Traduce Señas:

Installation Manual

Juan Mauricio Suaza Solórzano

Luis Alejandro Duarte Aldana

Juan Camilo Fierro Esquivel

Instructor: Karol Daniela Correa

November 4, 2026

Table of Contents

Introduction 4

1. Key Concepts 4

    1.1 General Characteristics (Ports) 4

    1.2 Database and Migrations 4

    1.3 Configuration Variables (Names Only) 7

2. Installation 8

    2.1 Requirements 8

    2.2 Getting the Code 8

    2.3 Installation With Docker (Recommended Route) 9

    2.4 Development Mode Without Docker 10

    2.5 Installation on a Production Server 11

3. Configuration and First Use 12

    3.1 Checking That It Works 12

    3.2 Creating and Activating the First Account 13

    3.3 First Administrator 14

    3.4 Resetting the Database (Deletes All Data) 14

4. Operation 14

5. Common Problems 15

References 17

Introduction

This manual explains how to get Traduce Señas running on a local computer with Docker Compose, and how to run it in development mode. It has been tested with Backend `v1.4.0` (`main` branch, commit `a962982`) and Frontend `v1.3.1` (`main` branch, commit `a5e231d`).

It does not include configuration values. You define the keys and passwords yourself, in your `.env` file, and they are **never** written in this manual or in the repository.

1. Key Concepts

1.1 General Characteristics (Ports)

Table 1

System Ports on the Computer

Component	Port	Published on the computer
api-gateway (single entry point to the API)	8080	Yes. Compose publishes it on all interfaces of the machine.
PostgreSQL for `iam`, `recognition`, `lexicon`, `analytics` and `profile`	5433, 5434, 5435, 5436 and 5437	Only on `127.0.0.1`
MongoDB (gesture samples)	27017	Only on `127.0.0.1`
RabbitMQ panel	15672	Only on `127.0.0.1`
Services `iam`, `recognition`, `lexicon`, `analytics` and `profile` (3001 to 3005) and AMQP (5672)	—	Not published: only reachable inside the Docker network
Expo development server (web)	8081	Not part of the compose file

To call the API from outside Docker, always use `http://localhost:8080/api/...`. If the network of the computer is not trusted, you can change the gateway port in `docker-compose.yml` to `127.0.0.1:8080:8080`.

1.2 Database and Migrations

Each service has its own database. The SQL files are in `services/<service>/db/`, and the compose file mounts them in each database container. PostgreSQL runs those files **only**

when the data volume is created for the first time. The full list of mounted files is in the Technical manual, section 4.2.

Practical consequences:

- On a new installation you do not have to do anything: the schema is created on the first `docker compose up`.
- If you already have volumes created with an earlier version, **do not delete them** to get the new migrations: apply them by hand, in order, with the platform running (`docker compose up -d`). The commands are below.
- If you delete the volumes with `docker compose down -v`, all the data is lost.

Migrations for a volume that already exists. Run them from the root of the Backend repository, in this order. Each file runs once, except `009_outbox_events.sql` in `iam`, which is idempotent and can be run again if it was applied with an earlier version.

```
docker compose exec -T postgres-iam       psql -U postgres -d iam
< services/iam-service/db/009_outbox_events.sql
docker compose exec -T postgres-iam       psql -U postgres -d iam
< services/iam-service/db/010_users_password_nullable.sql
docker compose exec -T postgres-iam       psql -U postgres -d iam
< services/iam-service/db/011_account_deletion_token.sql
docker compose exec -T postgres-iam       psql -U postgres -d iam
< services/iam-service/db/012_outbox_claim_token.sql
docker compose exec -T postgres-iam       psql -U postgres -d iam
< services/iam-service/db/013_users_terms_version.sql
docker compose exec -T postgres-analytics psql -U postgres -d analytics
< services/analytics-service/db/002_erased_users.sql
docker compose exec -T postgres-profile   psql -U postgres -d profile
< services/profile-service/db/002_erased_users.sql
docker compose exec -T postgres-profile   psql -U postgres -d profile
< services/profile-service/db/003_ui_languages.sql
docker compose exec -T postgres-profile   psql -X -v ON_ERROR_STOP=1 -U
postgres -d profile < services/profile-
service/db/004_achievement_catalog.sql
docker compose exec -T postgres-profile   psql -X -v ON_ERROR_STOP=1 -U
postgres -d profile < services/profile-
service/db/005_obtainable_achievements.sql
docker compose exec -T postgres-profile   psql -X -v ON_ERROR_STOP=1 -U
postgres -d profile < services/profile-
service/db/006_translation_milestones.sql
```

```
docker compose exec -T postgres-profile   psql -X -v ON_ERROR_STOP=1 -U
postgres -d profile < services/profile-
service/db/007_functional_achievements.sql
docker compose exec -T postgres-profile   psql -X -v ON_ERROR_STOP=1 -U
postgres -d profile < services/profile-service/db/008_admin_reports.sql
docker compose exec -T postgres-profile   psql -X -v ON_ERROR_STOP=1 -U
postgres -d profile < services/profile-
service/db/009_daily_challenges.sql
docker compose exec -T postgres-profile   psql -X -v ON_ERROR_STOP=1 -U
postgres -d profile < services/profile-
service/db/010_challenge_reward_achievements.sql
docker compose exec -T postgres-recognition psql -U postgres -d
recognition < services/recognition-service/db/003_erased_users.sql
docker compose exec -T postgres-recognition psql -X -v ON_ERROR_STOP=1
-U postgres -d recognition < services/recognition-
service/db/004_sign_templates_word_kind.sql
docker compose exec -T postgres-recognition psql -X -v ON_ERROR_STOP=1
-U postgres -d recognition < services/recognition-
service/db/005_translation_retention.sql
```

Notes on this list:

- This list is the complete one. The comment at the top of `docker-compose.yml` omits `iam-service/db/012_outbox_claim_token.sql` and `profile-service/db/007_functional_achievements.sql`, although both files are mounted.
- If you use PowerShell, the `<` operator does not work. Replace it with a pipe, for example: `Get-Content services/iam-service/db/009_outbox_events.sql | docker compose exec -T postgres-iam psql -U postgres -d iam`.
- Migrations 009 and 010 of `profile` add daily challenges and point-based medals. Before applying them, stop the service with `docker compose stop profile-service`, and start it again with `docker compose up -d profile-service` when you are done.
- Keep the order shown: the `iam` files depend on the earlier ones, and `009_outbox_events.sql` must exist before `iam` publishes `iam.UserDeleted`.

1.3 Configuration Variables (Names Only)

The `.env.example` file at the root of the Backend repository documents all the variables. For Docker Compose, copy that file to `.env` at the root of the Backend.

Table 2

Configuration Variables (Names Only)

Variable	Required	Purpose
`JWT_SECRET`	Yes	Shared secret used to sign and verify sessions. At least 32 characters. All services use the same one.
`POSTGRES_PASSWORD`	Yes	Password of the five PostgreSQL servers (user `postgres`).
`MONGO_ROOT_PASSWORD`	Yes	Password of the MongoDB user.
`RABBITMQ_PASSWORD`	Yes	Password of the RabbitMQ user.
`MONGO_ROOT_USER`, `RABBITMQ_USER`	No	Names of those users (default `signa`).
`CORS_ORIGINS`	No	Web origins (comma separated) allowed to call the API. By default, local Expo web.
`EMAIL_USER`, `EMAIL_PASS`	No	Gmail account that sends the codes and the email notices. Use an app password.
`GOOGLE_CLIENT_IDS`	No	Google OAuth client identifiers. If empty, Google sign-in responds with 503.
`FACEBOOK_APP_ID`, `FACEBOOK_APP_SECRET`	No	Facebook application details. If empty, Facebook sign-in responds with 503.
`RECOGNITION_TRANSLATION_RETENTION_DAYS`	No	Days that translations are kept. Default 30. With `0`, they are not deleted.
`ADMIN_REPORTS_ENABLED`, `APP_URL`	No	Turns on the emailed reports for administrators and sets the address of the button in those emails.
`DOMAIN`, `FRONTEND_DIR`, `CORS_ORIGINS_PROD`, `GATEWAY_TRUST_PROXY`	No	Production only (see section 2.5).

The database credentials are not in `.env`: the compose file sets them from `POSTGRES_PASSWORD`. The frontend has its own variable, `EXPO_PUBLIC_API_URL`, which is explained in section 2.3.

2. Installation

2.1 Requirements

Table 3

Requirements

You need	Details
Git	To clone the repositories.
Docker Desktop	With `docker` and `docker compose` available (Docker 24 or later and Compose 2.20 or later, according to the team's local setup guide).
Node.js 20 or later	Only for development mode without Docker (section 2.4) and for the frontend (`npm install`).
A terminal	PowerShell or Git Bash. The commands in section 1.2 that use < run in Git Bash.
Disk space	Several free GB for Docker images and dependencies.
Internet	To download images and dependencies, and for the browser to fetch MediaPipe when using the camera.
An email account to send codes	Optional. Without it, sign-up works, but the code does not arrive by email (see section 3.2).
Expo Go on a phone	Optional, to test the app on a phone.

2.2 Getting the Code

There are two public repositories, both on the `main` branch:

```
git clone -b main https://github.com/mauro-1508/backendTS.git
git clone -b main https://github.com/tadeo77789/frontendTS.git
```

Confirm the versions with `git -C backendTS rev-parse --short HEAD` (it must show `a962982`) and `git -C frontendTS rev-parse --short HEAD` (it must show `a5e231d`). If `main` has moved on, use the version indicated in the Technical manual.

The Backend repository contains `docker-compose.yml`, `docker-compose.prod.yml`, `.env.example` and the Dockerfile of each service. You do not need any file from another location.

2.3 Installation With Docker (Recommended Route)

Step A. Create the configuration file. From the root of the Backend repository:

```
cd backendTS
cp .env.example .env
```

In PowerShell, use `Copy-Item .env.example .env`. Open `.env` and fill in, at a minimum:

- `JWT_SECRET`: a text of 32 characters or more. You can generate it with `openssl rand -hex 32`.
- `POSTGRES_PASSWORD`, `MONGO_ROOT_PASSWORD` and `RABBITMQ_PASSWORD`: passwords of your own. Do not use the characters `@ : / ? #`, because they go inside the connection URLs.

If you want the codes to arrive by email, also fill in `EMAIL_USER` and `EMAIL_PASS`. Do not upload `.env` to any repository.

Step B. Build and start the platform:

```
docker compose up --build -d
docker compose ps
```

The first time it takes several minutes, because it builds the images. The compose file starts 13 containers: the gateway, the five services, the five PostgreSQL servers, MongoDB and RabbitMQ. Each service waits until its database is healthy, and the gateway waits until the five services respond. Repeat `docker compose ps` until every container shows `running` or `healthy`.

To stop without deleting data: `docker compose down`. To delete the data as well: `docker compose down -v` (see section 3.4).

Step C. Start the frontend. In another terminal, from the folder that contains the Backend repository (the one that holds `frontendTS`):

```
cd ../frontendTS
npm install
```

Create a `.env` file at the root of the frontend with this line:

```
EXPO_PUBLIC_API_URL=http://localhost:8080/api
```

To test from a phone on the same network, use the IP address of your computer instead of `localhost` (for example, `http://192.168.1.20:8080/api`). The variable is read at build time: if you change it, restart Expo.

```
npm run web
```

Expo opens the web app at `http://localhost:8081`. The backend already accepts that origin by default. For the mobile app, use `npm start` and Expo Go.

2.4 Development Mode Without Docker

It is useful for programming with automatic reload. You need Node.js 20 or later and the Docker databases, which is the simplest option because the compose file already creates their schema.

1. **Databases.** From the root of the Backend, start only the PostgreSQL servers:

   ```
   docker compose up -d postgres-iam postgres-recognition postgres-lexicon
   postgres-analytics postgres-profile
   ```

   They listen on `127.0.0.1`, on ports 5433 to 5437.

2. **Dependencies.** At the root of the Backend: `npm install`.

3. **Configuration of each service.** Copy the `.env.example` of each service to its `.env` (`services/api-gateway`, `services/iam-service`, `services/recognition-service`, `services/lexicon-service`, `services/analytics-service` and `services/profile-service`). Inside each file, the variables have the name without the prefix: `IAM_DB_HOST` becomes `DB_HOST`. Use the same `JWT_SECRET` in all of them and, in `DB_PASSWORD`, the

same value as `POSTGRES_PASSWORD`. With `RABBITMQ_URL` and `MONGO_URL` empty, events and samples are kept in memory and are lost on restart.

4. **Start.** From the root of the Backend: `npm run dev:all`. It starts the gateway and the five services with automatic reload. For one service only: `npm run dev -w @traduce/iam-service`.

If the full Docker stack is still running, port 8080 is taken by the gateway container. Stop it with `docker compose stop api-gateway`.

2.5 Installation on a Production Server

The production compose file is added to the base file with `-f`. From the root of the Backend:

```
docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d --build
```

Requirements and additional variables:

- The Frontend repository cloned next to the Backend, or the path set in `FRONTEND_DIR` (by default `../frontendTS`).
- `DOMAIN` (by default `traducesenas.tech`), with the DNS pointing to the server, and `www` redirected to the domain.
- Ports 80 and 443 open, also in the provider's panel. Caddy needs port 80 to obtain the certificate.
- `CORS_ORIGINS_PROD` only if you need to allow more web origins.

With this file the gateway stops being published on port 8080: everything enters through HTTPS. The full deployment procedure (backups, smoke tests and rollback) is in the Deployment plan (evidence 07).

Production server status: checked on November 4, 2026: HTTPS is active, HTTP redirects to HTTPS and `/api/health` answers `ok` with version 1.4.0 and all five services up.

3. Configuration and First Use

3.1 Checking That It Works

1. **Container status.** `docker compose ps`. The five services, the gateway and the databases must show `running` or `healthy`.
2. **Aggregated status.** From the computer:

   ```
   curl http://localhost:8080/health
   ```

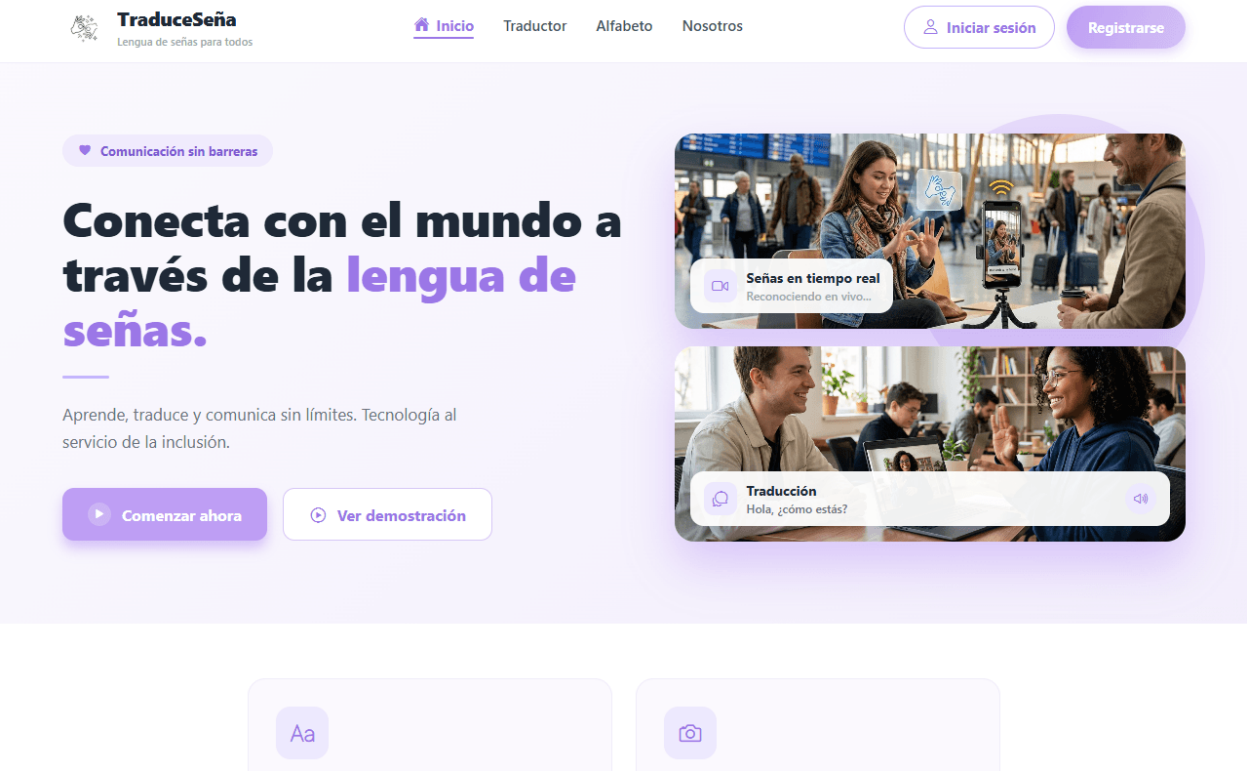
   The answer is HTTP 200 with `"status":"ok"` when all services respond. If one is down, the gateway still answers 200, but with `"status":"degraded"` and `"down"` for that service. When starting for the first time, `degraded` for a few seconds is normal: repeat the check.
3. **Functional check.** The gateway forwards the request to `iam` and validates it before doing anything else:

   ```
   curl -i -X POST http://localhost:8080/api/auth/login -H "Content-Type: application/json" -d "{}"
   ```

   It must answer `400` with `"code":"INVALID_BODY"`. That response proves the request reached `iam`.
4. **From the web.** Open `http://localhost:8081` (with `npm run web`). The home page must load.

Figure 1

Application Home Page



5. If something fails: `docker compose logs <service>`, for example `docker compose logs iam-service`.

3.2 Creating and Activating the First Account

1. On the web, tap **Sign up** and complete the **Create your Account** form. Accept the terms.
2. New accounts stay **inactive** until the email is confirmed with a code. If you configured `EMAIL_USER` and `EMAIL_PASS`, the code arrives by email. If not, activate the account by hand from the root of the Backend:

   ```
   docker compose exec -T postgres-iam psql -U postgres -d iam -c "UPDATE users SET status='ACTIVE' WHERE email='<email_in_lowercase>'"
   ```

   Emails are stored in lowercase, so write it that way in the query.

If sign-up says the code could not be sent, the account was still created: request a new code with **Resend** once the email is configured.

3.3 First Administrator

It is done with an SQL statement, only once: the `ADMIN` role is assigned to the email of the account. The step-by-step procedure is in the Administrator manual, section 3.

3.4 Resetting the Database (Deletes All Data)

This deletes ALL the data in all the databases (accounts, history, templates and samples). If it matters to you, make a backup first, for example with `docker compose exec -T postgres-iam pg_dump -U postgres iam > backup-iam.sql`, and repeat it for each database.

```
docker compose down -v
docker compose up --build -d
```

When the volumes are recreated, PostgreSQL runs the mounted SQL files again, so there is no need to apply migrations by hand.

4. Operation

Table 4

Operation

I want to...	Command
Stop everything without deleting data	`docker compose stop`
Start again	`docker compose up -d`
See the status	`docker compose ps`
Stop and remove containers (data is kept)	`docker compose down`
See the messages of one service	`docker compose logs -f iam-service`
Rebuild after changing code	`docker compose up --build -d`
Also delete the data	`docker compose down -v` (**deletes the data**)
Update the code	`git pull` in `backendTS` and in `frontendTS`, then `docker compose up --build -d`. New migrations are not applied automatically to a database that already exists: see section 1.2.

5. Common Problems

Table 5

Common Problems

Symptom	Likely cause	What to do
`docker compose up` fails with "Define JWT_SECRET", "POSTGRES_PASSWORD", "MONGO_ROOT_PASSWORD" or "RABBITMQ_PASSWORD"	That variable is missing in `.env`	Copy `.env.example` to `.env` at the root of the Backend and fill it in
The services do not start and `iam` keeps restarting	`JWT_SECRET` has fewer than 32 characters	Use a secret of 32 characters or more
`/health` shows `degraded` or `down` permanently	A database does not respond, RabbitMQ is not ready, or a password with special characters breaks the URL	Check `docker compose logs <service>`. Remove `@ : / ? #` from the passwords and recreate the volumes if a password changed
I changed a `.sql` file and nothing happened	PostgreSQL only runs the SQL files when the volume is created	Apply the migration by hand (section 1.2) or use `docker compose down -v` (deletes data)
The verification code does not arrive	`EMAIL_USER` or `EMAIL_PASS` are empty or rejected	Activate the account by hand (section 3.2) or configure a Gmail app password and restart `iam-service`
The browser blocks the calls (CORS)	The origin of your web app is not in `CORS_ORIGINS`	Add it to `CORS_ORIGINS` and restart the gateway with `docker compose up -d api-gateway`
The app does not connect to your backend	It points to the public API by default	Set `EXPO_PUBLIC_API_URL` in the frontend `.env` and restart Expo
Google or Facebook sign-in shows a not-configured notice	`GOOGLE_CLIENT_IDS` or `FACEBOOK_APP_ID` is missing in the backend, or the platform credential is missing in the frontend	Configure the identifiers on both sides
Port 8080 or 5433 to 5437 is in use	Another program or the Docker stack uses it	Close the other program or stop the container (`docker compose stop api-gateway`)

Symptom	Likely cause	What to do
`npm run dev:all` cannot connect to the database	The Docker PostgreSQL servers are not running	Run `docker compose up -d postgres-iam postgres-recognition postgres-lexicon postgres-analytics postgres-profile`
Expo does not start or hangs	Corrupted cache	Run `npx expo start -c`
All users lose their session after a restart	`JWT_SECRET` changed	Use a fixed `JWT_SECRET` in `.env`
The camera does not work on the web	The browser has no permission or the origin is not secure	Accept the permission. Some browsers only allow the camera on `localhost` or over HTTPS

References

Caddy Project. (n.d.). *Caddy documentation*. Retrieved November 4, 2026, from https://caddyserver.com/docs/

Docker Inc. (n.d.). *Docker Compose*. Docker Docs. Retrieved November 4, 2026, from https://docs.docker.com/compose/

Expo. (n.d.). *Expo documentation*. Retrieved November 4, 2026, from https://docs.expo.dev/

F5, Inc. (n.d.). *nginx documentation*. Retrieved November 4, 2026, from https://nginx.org/en/docs/

OpenJS Foundation. (n.d.). *Node.js documentation*. Retrieved November 4, 2026, from https://nodejs.org/docs/latest/api/

The PostgreSQL Global Development Group. (n.d.). *PostgreSQL 16 documentation*. Retrieved November 4, 2026, from https://www.postgresql.org/docs/16/